FastAPI Reference

Middleware

There are several middlewares available provided by Starlette directly.

Read more about them in the FastAPI docs for Middleware.

class fastapi.middleware.cors.CORSMiddleware

```
CORSMiddleware(
    app,
    allow_origins=(),
    allow_methods=("GET",),
    allow_headers=(),
    allow_credentials=False,
    allow_origin_regex=None,
    allow_private_network=False,
    expose_headers=(),
    max_age=600,
)
```

Ask AI

Source code in `starlette/middleware/cors.py`

```python
def __init__(
    self,
    app: ASGIApp,
    allow_origins: Sequence[str] = (),
    allow_methods: Sequence[str] = ("GET",),
    allow_headers: Sequence[str] = (),
    allow_credentials: bool = False,
    allow_origin_regex: str | None = None,
    allow_private_network: bool = False,
    expose_headers: Sequence[str] = (),
    max_age: int = 600,
) -> None:
    if "*" in allow_methods:
        allow_methods = ALL_METHODS

    compiled_allow_origin_regex = None
    if allow_origin_regex is not None:
        compiled_allow_origin_regex = re.compile(allow_origin_regex)

    allow_all_origins = "*" in allow_origins
    allow_all_headers = "*" in allow_headers
    preflight_explicit_allow_origin = not allow_all_origins or allow_credentials

    simple_headers: dict[str, str] = {}
    if allow_all_origins:
        simple_headers["Access-Control-Allow-Origin"] = "*"
    if allow_credentials:
        simple_headers["Access-Control-Allow-Credentials"] = "true"
    if expose_headers:
        simple_headers["Access-Control-Expose-Headers"] = ", ".join(expose_headers)

    preflight_headers: dict[str, str] = {}
    if preflight_explicit_allow_origin:
        # The origin value will be set in preflight_response() if it is allowed.
        preflight_headers["Vary"] = "Origin"
    else:
        preflight_headers["Access-Control-Allow-Origin"] = "*"
    preflight_headers.update(
        {
            "Access-Control-Allow-Methods": ", ".join(allow_methods),
            "Access-Control-Max-Age": str(max_age),
        }
    )
    allow_headers = sorted(SAFELISTED_HEADERS | set(allow_headers))
    if allow_headers and not allow_all_headers:
        preflight_headers["Access-Control-Allow-Headers"] = ", ".join(allow_headers)
    if allow_credentials:
        preflight_headers["Access-Control-Allow-Credentials"] = "true"

    self.app = app
    self.allow_origins = allow_origins
    self.allow_methods = allow_methods
    self.allow_headers = [h.lower() for h in allow_headers]
    self.allow_all_origins = allow_all_origins
    self.allow_all_headers = allow_all_headers
    self.preflight_explicit_allow_origin = preflight_explicit_allow_origin
    self.allow_origin_regex = compiled_allow_origin_regex
    self.allow_private_network = allow_private_network
    self.simple_headers = simple_headers
    self.preflight_headers = preflight_headers
```

attr app `instance-attribute`

```python
app = app
```

attr allow_origins `instance-attribute`

```
allow_origins = allow_origins
```

attr allow_methods `instance-attribute`

```
allow_methods = allow_methods
```

attr allow_headers `instance-attribute`

```
allow_headers = [(lower()) for h in allow_headers]
```

attr allow_all_origins `instance-attribute`

```
allow_all_origins = allow_all_origins
```

attr allow_all_headers `instance-attribute`

```
allow_all_headers = allow_all_headers
```

attr preflight_explicit_allow_origin `instance-attribute`

```
preflight_explicit_allow_origin = (
    preflight_explicit_allow_origin
)
```

attr allow_origin_regex `instance-attribute`

```
allow_origin_regex = compiled_allow_origin_regex
```

attr allow_private_network `instance-attribute`

```
allow_private_network = allow_private_network
```

attr simple_headers `instance-attribute`

```
simple_headers = simple_headers
```

attr preflight_headers `instance-attribute`

```
preflight_headers = preflight_headers
```

meth is_allowed_origin

```
is_allowed_origin(origin)
```

Source code in `starlette/middleware/cors.py`

```python
def is_allowed_origin(self, origin: str) -> bool:
    if self.allow_all_origins:
        return True

    if self.allow_origin_regex is not None and self.allow_origin_regex.fullmatch(origin):
        return True

    return origin in self.allow_origins
```

meth preflight_response

```python
preflight_response(request_headers)
```

Source code in `starlette/middleware/cors.py`

```python
def preflight_response(self, request_headers: Headers) -> Response:
    requested_origin = request_headers["origin"]
    requested_method = request_headers["access-control-request-method"]
    requested_headers = request_headers.get("access-control-request-headers")
    requested_private_network = request_headers.get("access-control-request-private-network")

    headers = dict(self.preflight_headers)
    failures: list[str] = []

    if self.is_allowed_origin(origin=requested_origin):
        if self.preflight_explicit_allow_origin:
            # The "else" case is already accounted for in self.preflight_headers
            # and the value would be "*".
            headers["Access-Control-Allow-Origin"] = requested_origin
    else:
        failures.append("origin")

    if requested_method not in self.allow_methods:
        failures.append("method")

    # If we allow all headers, then we have to mirror back any requested
    # headers in the response.
    if self.allow_all_headers and requested_headers is not None:
        headers["Access-Control-Allow-Headers"] = requested_headers
    elif requested_headers is not None:
        for header in [h.lower() for h in requested_headers.split(",")]:
            if header.strip() not in self.allow_headers:
                failures.append("headers")
                break

    if requested_private_network is not None:
        if self.allow_private_network:
            headers["Access-Control-Allow-Private-Network"] = "true"
        else:
            failures.append("private-network")

    # We don't strictly need to use 400 responses here, since its up to
    # the browser to enforce the CORS policy, but its more informative
    # if we do.
    if failures:
        failure_text = "Disallowed CORS " + ", ".join(failures)
        return PlainTextResponse(failure_text, status_code=400, headers=headers)

    return PlainTextResponse("OK", status_code=200, headers=headers)
```

meth simple_response `async`

```
simple_response(scope, receive, send, request_headers)
```

Source code in `starlette/middleware/cors.py`

```
151 async def simple_response(self, scope: Scope, receive: Receive, send: Send, request_headers:
152 Headers) -> None:
153     send = functools.partial(self.send, send=send, request_headers=request_headers)
        await self.app(scope, receive, send)
```

meth send `async`

```
send(message, send, request_headers)
```

Source code in `starlette/middleware/cors.py`

```
155 async def send(self, message: Message, send: Send, request_headers: Headers) -> None:
156     if message["type"] != "http.response.start":
157         await send(message)
158         return
159
160     message.setdefault("headers", [])
161     headers = MutableHeaders(scope=message)
162     headers.update(self.simple_headers)
163     origin = request_headers["Origin"]
164     has_cookie = "cookie" in request_headers
165
166     # If request includes any cookie headers, then we must respond
167     # with the specific origin instead of '*'.
168     if self.allow_all_origins and has_cookie:
169         self.allow_explicit_origin(headers, origin)
170
171     # If we only allow specific origins, then we have to mirror back
172     # the Origin header in the response.
173     elif not self.allow_all_origins and self.is_allowed_origin(origin=origin):
174         self.allow_explicit_origin(headers, origin)
175
176     await send(message)
```

meth allow_explicit_origin `staticmethod`

```
allow_explicit_origin(headers, origin)
```

Source code in `starlette/middleware/cors.py`

```
178 @staticmethod
179 def allow_explicit_origin(headers: MutableHeaders, origin: str) -> None:
180     headers["Access-Control-Allow-Origin"] = origin
181     headers.add_vary_header("Origin")
```

It can be imported from `fastapi`:

```
from fastapi.middleware.cors import CORSMiddleware
```

class fastapi.middleware.gzip.GZipMiddleware

```
GZipMiddleware(app, minimum_size=500, compresslevel=9)
```

Source code in `starlette/middleware/gzip.py`

```
def __init__(self, app: ASGIApp, minimum_size: int = 500, compresslevel: int = 9) -> None:
    self.app = app
    self.minimum_size = minimum_size
    self.compresslevel = compresslevel
```

attr app `instance-attribute`

```
app = app
```

attr minimum_size `instance-attribute`

```
minimum_size = minimum_size
```

attr compresslevel `instance-attribute`

```
compresslevel = compresslevel
```

It can be imported from `fastapi`:

```
from fastapi.middleware.gzip import GZipMiddleware
```

class fastapi.middleware.httpsredirect.HTTPSRedirectMiddleware

```
HTTPSRedirectMiddleware(app)
```

Source code in `starlette/middleware/httpsredirect.py`

```
def __init__(self, app: ASGIApp) -> None:
    self.app = app
```

attr app `instance-attribute`

```
app = app
```

It can be imported from `fastapi`:

```
from fastapi.middleware.httpsredirect import HTTPSRedirectMiddleware
```

class fastapi.middleware.trustedhost.TrustedHostMiddleware

```
TrustedHostMiddleware(
    app, allowed_hosts=None, www_redirect=True
)
```

Source code in `starlette/middleware/trustedhost.py`

```
def __init__(
    self,
    app: ASGIApp,
    allowed_hosts: Sequence[str] | None = None,
    www_redirect: bool = True,
) -> None:
    if allowed_hosts is None:
        allowed_hosts = ["*"]

    for pattern in allowed_hosts:
        assert "*" not in pattern[1:], ENFORCE_DOMAIN_WILDCARD
        if pattern.startswith("*") and pattern != "*":
            assert pattern.startswith("*."), ENFORCE_DOMAIN_WILDCARD
    self.app = app
    self.allowed_hosts = list(allowed_hosts)
    self.allow_any = "*" in allowed_hosts
    self.www_redirect = www_redirect
```

attr app `instance-attribute`

```
app = app
```

attr allowed_hosts `instance-attribute`

```
allowed_hosts = list(allowed_hosts)
```

attr allow_any `instance-attribute`

```
allow_any = '*' in allowed_hosts
```

attr www_redirect `instance-attribute`

```
www_redirect = www_redirect
```

It can be imported from `fastapi`:

```
from fastapi.middleware.trustedhost import TrustedHostMiddleware
```